

Hands-On: React Context

Step 1: Creating a Context

The first step in using React Context is to create a context. To create a context, you can use the `React.createContext()` method. In this example, we will create a context for a user:

```
import React from 'react';

const UserContext = React.createContext();
```

Step 2: Providing a value with Context Provider

After creating the context, you need to provide a value to it. You can do this by wrapping the components that need to access the context in a `Context.Provider` component.

```
import React from 'react';

const UserContext = React.createContext();

function App() {
  const user = {
    name: 'John',
    email: 'john@example.com'
  }

  return (
    <UserContext.Provider value={user}>
      <div className="App">
        <Header />
        <Content />
      </div>
    </UserContext.Provider>
  );
}
```

In this example, the `UserContext.Provider` component wraps the `Header` and `Content` components. The `value` prop provides the value to the context, which in this case is an object containing the user's name and email.

Step 3: Accessing Context in Components

To access the value of the context, you need to use the `useContext()` hook. This hook takes the context object as its argument and returns its current value.

```
import React, { useContext } from 'react';

const UserContext = React.createContext();

function Header() {
  const user = useContext(UserContext);

  return (
```

```

    <header>
      <h1>{user.name}'s Profile</h1>
      <p>{user.email}</p>
    </header>
  );
}

```

In this example, the **`Header`** component uses the **`useContext()`** hook to get the value of the **`UserContext`**. The **`user`** variable contains the user object that was passed as a value to the context.

Step 4: Updating Context

You can also update the context from a child component by passing a function as the value prop. The function takes the current value of the context as its argument and returns a new value.

```

import React, { useContext } from 'react';

const UserContext = React.createContext();

function Content() {
  const { name, setName } = useContext(UserContext);

  function handleNameChange(event) {
    setName(event.target.value);
  }

  return (
    <div>
      <h2>Edit Profile</h2>
      <form>
        <label>
          Name:
          <input type="text" value={name} onChange={handleNameChange} />
        </label>
      </form>
    </div>
  );
}

function App() {
  const [name, setName] = useState('John');

  return (
    <UserContext.Provider value={{ name, setName }}>
      <div className="App">
        <Header />
        <Content />
      </div>
    </UserContext.Provider>
  );
}

```

In this example, the `Content` component uses the `useContext()` hook to get the `name` and `setName` properties from the `UserContext`. The `handleNameChange` function updates the `name` property when the input field changes.

Conclusion

In this lab, you learned how to create a context in React and use it to share data between components. You also learned how to update the context from a child component using a function as the value prop. With this knowledge, you can now create more complex React applications that rely on data shared between multiple components.